K. S. School of Engineering and Management

Title of the Project:

CNN-Based Gesture Controlled Game Application

Course: Deep Learning & Reinforcement Learning - BAI 701

Submitted by:

Student Name	USN	Semester & Section	Team Number
Aryan Kumar	1KG23AD002	VII & "C"	2
Suman G.	1KG23AD054	VII & "C"	2
Tanmay Singh	1KG23AD056	VII & "C"	2

Submitted to:

Faculty Name: Mrs. Padma Priya K.

Department: Artificial Intelligence and Data Science

Academic Year: 2026-2027

1. ABSTRACT

This project is an interactive game that is played with hand gestures instead of a keyboard. A convolutional neural network (CNN) looks at the player's hand through an ordinary webcam and recognises one of four gestures: a closed fist, an open palm, a thumbs-up or a thumbs-down. Each gesture is turned into a game action that moves Pac-Man left, right, up or down in a maze game we wrote ourselves.

We trained the network on 2,000 hand images (500 per gesture) cropped from the public HaGRID dataset, split into 1,600 training, 200 validation and 200 test images. Rather than building a CNN from scratch, we adapted MobileNetV2 pretrained on ImageNet by giving it a new four-class output layer and training it in two stages. The final model reached 99.5% validation accuracy and 99.0% test accuracy (198 of 200 images), with a macro F1-score of 0.990, and 99.6% on 2,000 images of people who are not in our dataset.

In the game, recognition runs on its own thread at about 30 frames per second while the game keeps 60. A gesture only becomes a move when the model is at least 90% sure and the same gesture wins three of the last five frames. In live trials all 80 held gestures were read correctly and none of the 40 trials without a gesture moved Pac-Man.

Keywords: convolutional neural network, MobileNetV2, transfer learning, hand gesture recognition, real-time game control, Pygame.

2. INTRODUCTION

2.1 Background

Games are usually played with a keyboard, mouse or gamepad. Gesture control lets the player use their own hand instead, which feels more natural and needs nothing more than the webcam most laptops already have. The hard part is turning camera images into commands that are right almost every time.

CNNs are well suited to this because they learn visual features such as edges, textures and shapes directly from labelled images. Training one from scratch needs a very large dataset, but with *transfer learning* a network already trained on ImageNet [1] can be adapted to a new task with a few thousand images. We used MobileNetV2 [2], a CNN designed to run quickly on modest hardware, since the model has to process webcam frames many times a second while the game is running.

2.2 Problem Statement

Develop an interactive game application that uses a Convolutional Neural Network (CNN) to recognize hand gestures captured through a webcam and translate them into corresponding game actions.

To be playable, the system has to recognise the gestures reliably for different people and lighting, keep up with the camera without slowing the game below 60 frames per second, and do nothing when it is unsure, because a wrong turn into a ghost is worse than a missed one.

2.3 Objectives

- **Objective 1**: Prepare a balanced image dataset of four hand gestures from a documented public source.
- **Objective 2**: Build a CNN classifier for the four gestures using a pretrained MobileNetV2.
- **Objective 3**: Train the model and evaluate it on held-out test images with accuracy, precision, recall, F1-score and a confusion matrix.
- **Objective 4**: Develop a Pac-Man-style maze game in Pygame.

- **Objective 5**: Integrate the trained CNN with the game in real time, with the keyboard kept as a backup.

3. SYSTEM REQUIREMENTS

3.1 Hardware Requirements

Table 1 shows the laptop the project was developed and tested on, along with what is needed to run it.

Table 1: Hardware Requirements

Component	Used for development	Needed to run the game
Processor	Intel Core i7-11800H, 2.30 GHz	Any 64-bit processor
Graphics	NVIDIA GeForce RTX 3050 Ti Laptop GPU, 4 GB	Optional; the model also runs on the CPU. Training needs an NVIDIA CUDA GPU.
Memory	15.7 GB RAM	About 1 GB for the recognition process
Camera	Built-in laptop webcam (640 × 480)	Any webcam readable by OpenCV
Display	Laptop screen	At least 992 × 704 pixels

3.2 Software Requirements

Table 2 lists the software and libraries used in the project.

Table 2: Software Requirements

Software	Version	Used for
Operating system	Windows 11, 64-bit	Development and testing
Python	3.12.10	Programming language
PyTorch	2.14.0 (CUDA 13.0)	Building, training and running the CNN [3]
torchvision	0.29.0	MobileNetV2 and image transforms [4]
OpenCV	5.0.0	Webcam capture and image processing [5]
Pygame	2.6.1	Game window, drawing and input [6]

Software	Version	Used for
NumPy, Matplotlib, scikit-learn	2.5.3, 3.11.2, 1.9.1	Arrays, graphs and evaluation metrics [7], [8], [9]

4. PROPOSED SYSTEM

The proposed system is a maze game in which a trained CNN watching the player's hand decides which way Pac-Man turns. The keyboard keeps working the whole time, so the game is playable even without a camera.

4.1 Game Description

The game is our own version of the arcade classic Pac-Man, written from scratch in Pygame. Everything on screen is drawn with simple shapes, so no assets from the original game are used. Pac-Man moves through a 28 × 31 maze eating 330 pellets and 4 power pellets while four ghosts chase him. A power pellet frightens the ghosts for a few seconds so Pac-Man can eat them for bonus points. The player has three lives, and clearing the board starts a harder round. The window is 992 × 704 pixels, with the maze on the left and a gesture panel on the right that shows the live camera view and the recognised command.

4.2 Role of CNN

The CNN reads the player's hand. For every camera frame it receives a 300 × 300-pixel square where the player holds their hand and gives a score for each of the four gestures. The game never looks at the camera itself; it only receives the directions produced from these scores. Table 3 shows the mapping.

Table 3: Gesture-to-Action Mapping

Hand gesture	Dataset label	Class index	Game action
Closed fist	fist	0	Move LEFT
Open palm	palm	1	Move RIGHT
Thumbs up	like	2	Move UP
Thumbs down	dislike	3	Move DOWN

4.3 System Architecture

As Figure 1 shows, the application runs as two parts side by side. A recognition thread captures webcam frames, cuts out the hand region, runs the CNN and applies a confidence threshold and smoothing. The main thread runs the game at 60 frames per second. The two share a single snapshot that always holds only the newest result, so the game never has to process old predictions. Once per frame, a gesture controller reads this snapshot and passes any stable command to the game, through the same interface the keyboard uses.

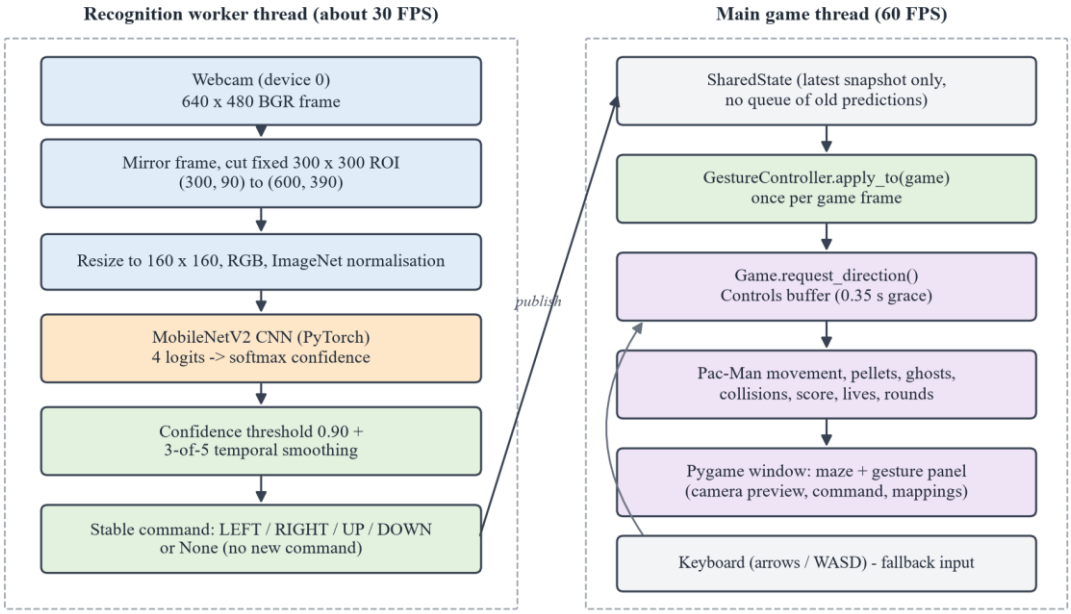


Figure 1: Proposed CNN-Based Gaming Application Architecture

5. DATASET DESCRIPTION

We used **HaGRID** (HAnd Gesture Recognition Image Dataset) by Kapitanov et al. [10], a large public collection of photographs of many different people making hand gestures, each with an official bounding box around the hand. It already contains our four gestures under the labels fist, palm, like and dislike. The images were taken from the *hagrid-sample-500k-384p* version on Hugging Face [11], which is released under the CC BY-SA 4.0 licence [12].

Since the game only shows the network a square around the hand, we cropped every photograph to its annotated hand box with 25% padding, made the crop square and saved it as a JPEG. Thumbs-up and thumbs-down images were taken from their own HaGRID classes; one was never made by flipping the other, because that would give a wrong label. The result is a

balanced dataset of 2,000 images, split per class with a fixed random seed (42) as shown in Table 4.

Table 4: Dataset Distribution

Class	Gesture	Images	Training	Validation	Test
left	Closed fist	500	400	50	50
right	Open palm	500	400	50	50
up	Thumbs up	500	400	50	50
down	Thumbs down	500	400	50	50
Total		**2,000**	**1,600**	**200**	**200**

Figure 2 shows sample training images. They vary a lot in background, lighting, skin tone and camera quality, which helps the model work with an unfamiliar webcam.



Figure 2: Sample Training Images from the Four Gesture Classes

6. DATA PREPROCESSING

Every image is resized to 160 × 160 pixels and normalised with the ImageNet mean and standard deviation, so it matches what the pretrained network expects. Training images are also

changed slightly at random (augmentation) so the model learns the gesture rather than details of particular photos. Table 5 lists the steps.

Table 5: Preprocessing Steps

Step	Training images	Validation, test and webcam images
Resize	160 × 160	160 × 160
Horizontal flip	Random, 50% chance	None
Rotation, shift, scale	Up to ±10°, 5% shift, 0.9 to 1.1 scale	None
Brightness and contrast	Changed by up to ±20%	None
Normalisation	ImageNet mean and standard deviation	Same

A horizontal flip does not change what a gesture means, but a vertical flip would turn a thumbs-up into a thumbs-down, so vertical flips are never used and rotation is limited to 10 degrees. Validation, test and webcam images get no random changes, so the game processes webcam frames exactly as the test images were processed. In the game, each frame is mirrored first and the fixed 300 × 300 hand region is cut out before this step.

7. CNN MODEL DESIGN

We used **MobileNetV2** [2] from torchvision with weights pretrained on ImageNet. Its convolutional feature extractor is reused, and its 1,000-class output layer is replaced by a new layer for our 4 classes. The model has 2,228,996 parameters in total.

7.1 CNN Architecture

Table 6 lists the layers in the order they run, with the output shape for one 160 × 160 input image. The shapes were taken from the actual model. MobileNetV2 groups most of its convolutions into *inverted residual blocks*, and *t* is the expansion factor of a block.

Table 6: CNN Architecture

Layer	Configuration	Output Shape
Input	RGB image	3 × 160 × 160

Layer	Configuration	Output Shape
Conv2D + ReLU6	32 filters, 3 × 3, stride 2	32 × 80 × 80
Inverted residual blocks 1 to 3	t = 1 and 6, 16 and 24 channels	24 × 40 × 40
Inverted residual blocks 4 to 10	t = 6, 32 and 64 channels	64 × 10 × 10
Inverted residual blocks 11 to 17	t = 6, 96, 160 and 320 channels	320 × 5 × 5
Conv2D + ReLU6	1,280 filters, 1 × 1	1280 × 5 × 5
Average pooling	Adaptive average pooling to 1 × 1	1280 × 1 × 1
Flatten	Reshape to a vector	1280
Dropout	p = 0.2 (training only)	1280
Dense (output)	1,280 inputs to 4 classes, softmax for confidence	4

7.2 Explanation of Layers

Convolution Layer

A convolution layer slides small learnable filters over the image to produce feature maps. Early layers detect edges and textures, deeper ones detect shapes such as fingers. MobileNetV2 has 52 convolution layers. Most of them sit in inverted residual blocks, which split a normal convolution into a 1 × 1 expansion, a 3 × 3 depthwise convolution and a 1 × 1 projection. This needs far fewer calculations, which is why the model is fast enough for real-time use. Each convolution is followed by batch normalisation [13].

ReLU Activation

MobileNetV2 uses ReLU6, f(x) = min(max(0, x), 6). Like ReLU it passes positive values and sets negative values to zero, but it also caps them at 6. It adds the non-linearity that lets the network learn complex shapes. The projection convolution at the end of each block has no activation, since ReLU there would throw away information [2].

Pooling Layer

Instead of max pooling, MobileNetV2 shrinks the feature maps with stride-2 convolutions (160 → 80 → 40 → 20 → 10 → 5). At the end, adaptive average pooling replaces each 5 × 5 feature

map by its average. This keeps how strongly each feature is present while ignoring exactly where the hand is.

Flatten Layer

The pooled output of 1280 × 1 × 1 values is flattened into a vector of 1,280 numbers so it can be fed to the dense layer.

Dense Layer

The dense (fully connected) layer connects all 1,280 features to the 4 outputs, giving 5,124 trainable parameters. It is the only completely new layer. A dropout layer before it switches off 20% of the features at random during training, which reduces overfitting [14].

Output Layer

The output layer gives four scores in the order left, right, up, down. Softmax turns them into probabilities, and the game accepts the most likely gesture only if its probability is 0.90 or higher. The model has no "no gesture" class; a low confidence simply means no new command.

7.3 CNN Architecture Diagram

Figure 3 shows the flow of data through the network and which parts were frozen or trained (Section 8).

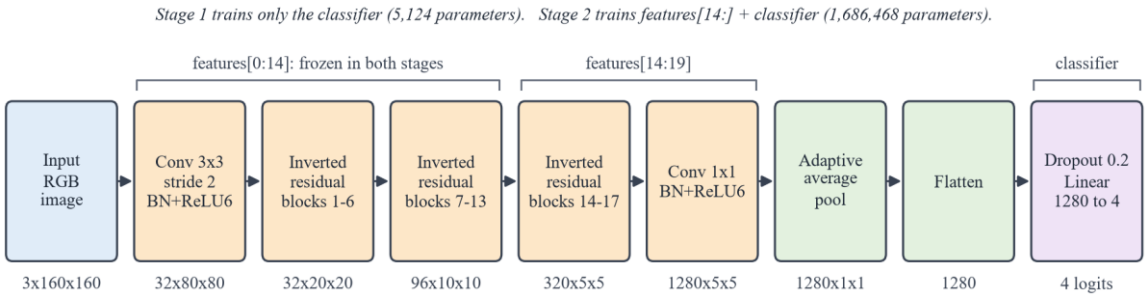


Figure 3: MobileNetV2 Architecture Adapted for Four Gesture Classes

8. MODEL TRAINING

The model was trained on the 1,600 training images, and the 200 validation images were used to pick the best checkpoint. The test images were never used during training. Training was

done in two stages. In stage 1 the pretrained layers were frozen and only the new output layer (5,124 parameters) was trained, so it could learn a sensible mapping before any pretrained weights changed. In stage 2 the last five feature blocks were unfrozen as well (1,686,468 trainable parameters) and trained with a ten times smaller learning rate, so the pretrained features were adjusted gently towards hands. Table 7 lists the settings.

Table 7: Training Parameters

Parameter	Value
Epochs	13 (5 in stage 1, 8 in stage 2)
Batch size	32
Optimizer	AdamW [15], weight decay 0.0001
Learning rate	0.001 in stage 1, 0.0001 in stage 2
Loss function	Cross-entropy
Activation functions	ReLU6 + Softmax
Training/validation split	1,600 / 200 images (plus 200 test images kept aside)
Input size	160 × 160 RGB
Early stopping	Patience of 3 epochs in stage 2 (not triggered)
Mixed precision	Yes, float16 on the GPU [16]
Training time	62.96 seconds on the RTX 3050 Ti

After every epoch the model was checked on the validation set and saved whenever the validation loss improved. The best model came from the final epoch and is stored as *model/best_direction_model.pt*.

9. MODEL PERFORMANCE EVALUATION

All results below come from the training history and the test evaluation that the project saved at the time. The test set was evaluated once, after training was finished.

9.1 Training Accuracy

During stage 1, with only the new layer learning, training accuracy rose from 67.0% to 88.7%. Once the last feature blocks were unfrozen it climbed quickly to 93.5% and finished at 99.81%, while the training loss fell from 0.922 to 0.0083.

9.2 Validation Accuracy

Validation accuracy went from 84.0% to 88.5% in stage 1 and reached 97.5% after the first fine-tuning epoch. It ended at 99.5% (199 of 200 images) in the final epoch, which also had the lowest validation loss (0.0362), so that model was kept.

9.3 Test Accuracy

On the 200 unseen test images the model got 198 right, a **test accuracy of 99.0%**, with a test loss of 0.0292 and a macro F1-score of 0.990. Table 8 gives the result for each class.

Table 8: Per-class Test Results

Class	Precision	Recall	F1-score	Images
left (fist)	1.000	1.000	1.000	50
right (palm)	1.000	0.980	0.990	50
up (thumbs up)	0.980	1.000	0.990	50
down (thumbs down)	0.980	0.980	0.980	50
Average	**0.990**	**0.990**	**0.990**	200

Some people in the test set also appear in the training set, so we also tested the model on 2,000 extra HaGRID images from 1,727 people who are not in our dataset at all. It scored 99.6% (1,992 of 2,000), which shows that it works on new people too.

9.4 Training and Validation Accuracy Graph

Figure 4 shows accuracy against epoch. The jump after the dashed line shows how much fine-tuning helped, and the two curves end close together, so the model did not overfit.

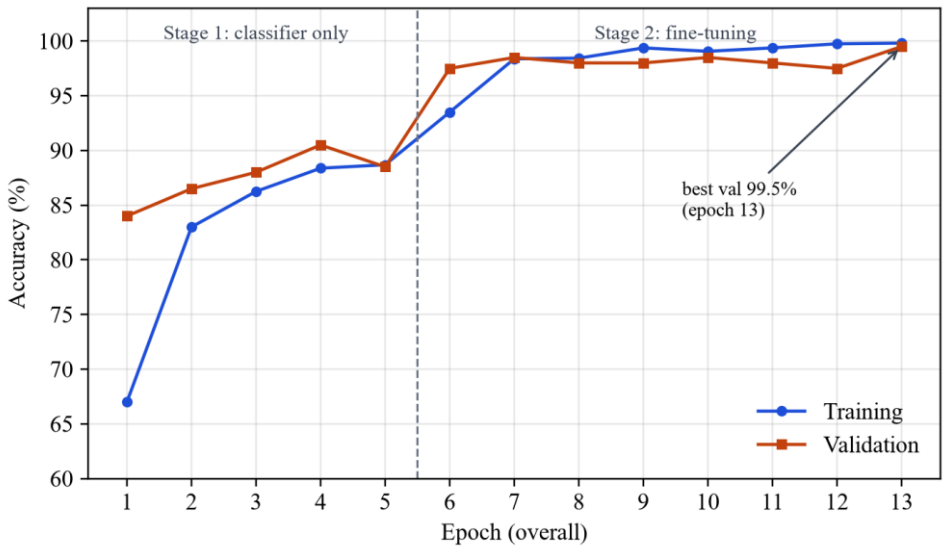


Figure 4: Training and Validation Accuracy Graph (Epoch vs Accuracy)

9.5 Training and Validation Loss Graph

Figure 5 shows loss against epoch. Both losses fall steadily and drop sharply when fine-tuning starts. The validation loss is lowest at the last epoch, where the model was saved.

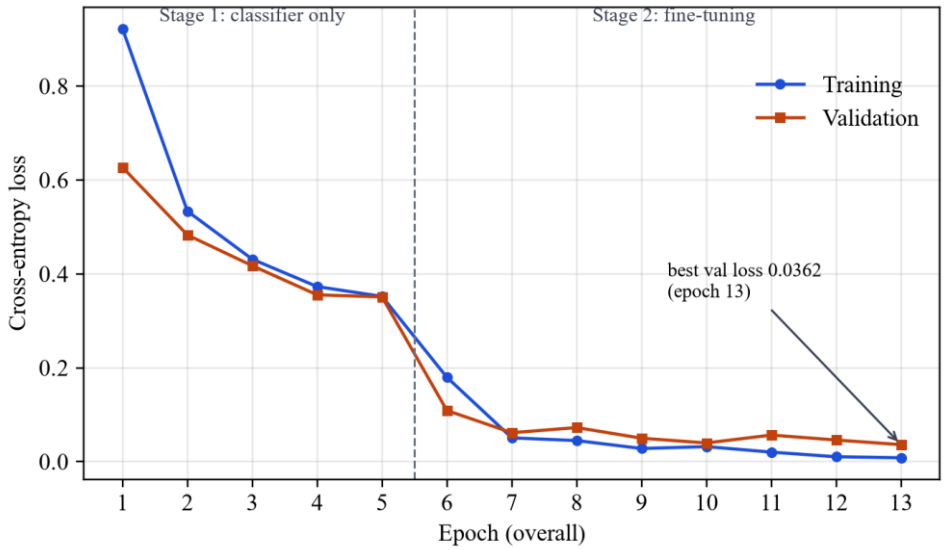


Figure 5: Training and Validation Loss Graph (Epoch vs Loss)

9.6 Confusion Matrix

Figure 6 shows the confusion matrix for the test set. All fists and thumbs-ups were correct. The two mistakes were an open palm read as thumbs-down (confidence 0.60) and a thumbs-down read as thumbs-up (confidence 0.90). In the game, the first would be ignored because it is below

the 0.90 threshold, and a single frame like the second cannot turn Pac-Man because of the smoothing.

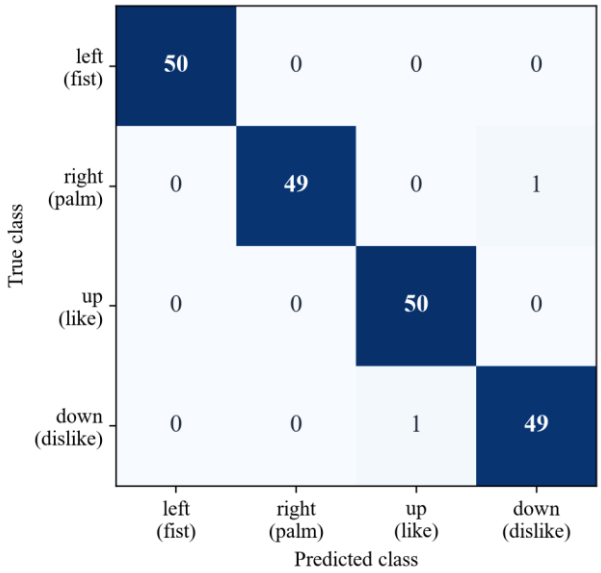


Figure 6: Confusion Matrix on the Test Set

9.7 Sample Predictions

Figure 7 shows real test images with their true and predicted labels. The top row shows confident correct predictions, and the bottom row shows the two least confident correct predictions and the two mistakes (in red).

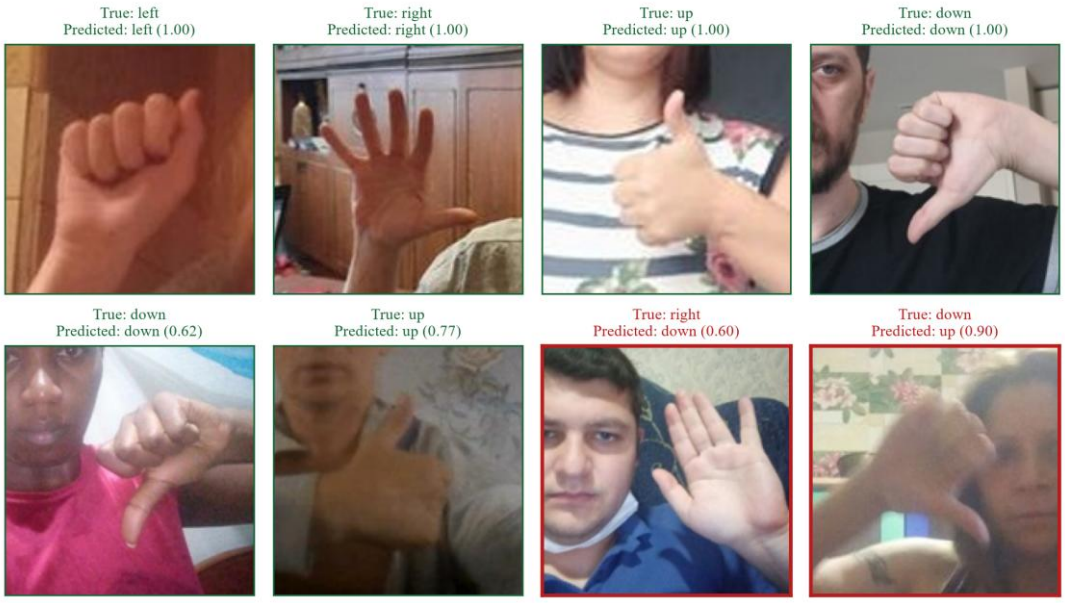


Figure 7: Sample Test Predictions (green: correct, red: wrong; confidence in brackets)

10. GAME DEVELOPMENT

The trained CNN was built into the game through a separate recognition thread and a gesture controller, described below.

10.1 Game Interface

The start screen (Figure 8) explains the gestures and keys while the model loads in the background, and the game starts when the player presses SPACE or ENTER.

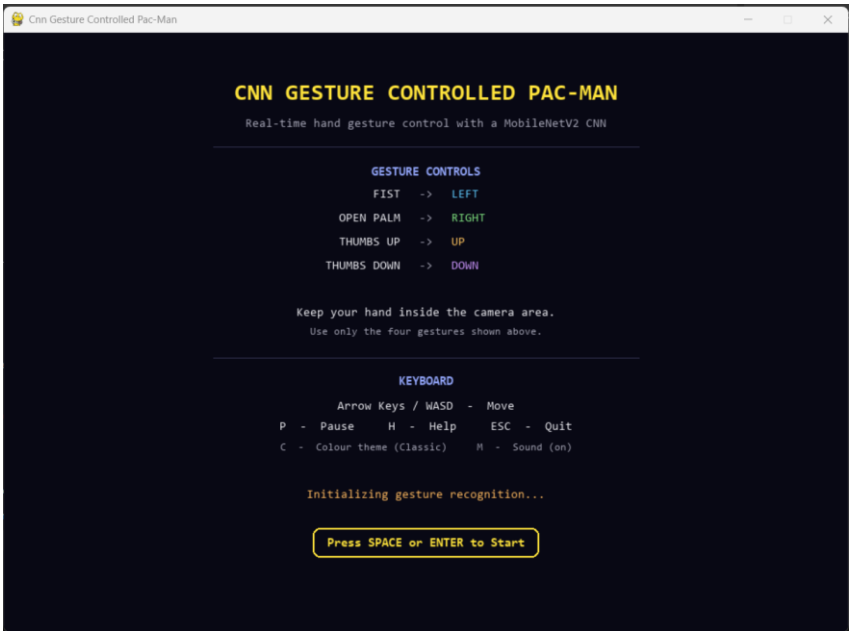


Figure 8: Start Screen of the Game

During play (Figure 9), the maze is on the left with the score, round and lives. The panel on the right shows a large live view of the hand region exactly as the CNN sees it, the current gesture with its confidence, the gesture-to-action table with the active row highlighted, the gesture-control status and the keyboard keys.

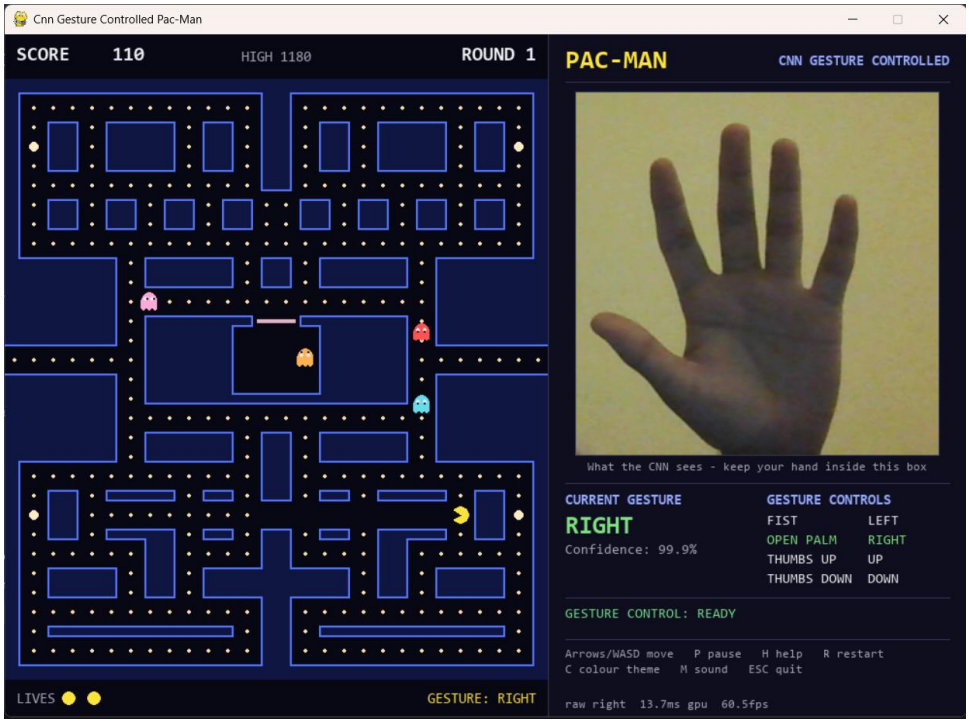


Figure 9: Gameplay with the Live Camera View and Gesture Panel

10.2 Game Logic

The sequence of events in the game is:

1. The start screen is shown while the CNN loads. The player presses SPACE or ENTER to begin.
2. A short "READY!" pause is shown, and then play starts.
3. Every frame (60 per second), the game reads keyboard input and the latest gesture command, moves Pac-Man and the ghosts, and draws the screen.
4. Pac-Man eats pellets for points. A power pellet frightens the ghosts so they can be eaten for bonus points.
5. If a ghost catches Pac-Man, he loses a life and everyone returns to their starting positions.
6. When all pellets are eaten, a new round starts with faster ghosts.
7. When no lives are left, the game is over. Pressing R starts a new game and ESC quits.

A direction request stays active for 0.35 seconds, so the player can show a gesture slightly before reaching a junction and Pac-Man still turns there. The main scoring rules are listed in Table 9.

Table 9: Scoring Rules

Item	Points / value
Pellet	10 points
Power pellet	50 points
Eating ghosts in a row	200, 400, 800, 1,600 points
Bonus fruit	100 to 5,000 points, depending on the round
Lives	3, plus one extra life at 10,000 points

10.3 CNN-Game Integration

The trained CNN communicates with the game in the following steps:

1. The recognition thread reads a 640 × 480 frame from the webcam.
2. The frame is mirrored and the 300 × 300 hand region is cut out.
3. The region is resized to 160 × 160 and normalised, exactly as in training.
4. The CNN predicts the four gesture probabilities.
5. A gesture is accepted only if its probability is at least 0.90 and it wins 3 of the last 5 frames.
6. The accepted gesture is mapped to a direction and saved in the shared snapshot.
7. Every game frame, the controller reads the snapshot and sends the direction to the game.

If no gesture is accepted, nothing is sent and Pac-Man keeps moving in his current direction. Because the game only reads the latest snapshot, recognition never slows the game down. Table 10 lists the main settings.

Table 10: Recognition Settings

Setting	Value
Camera frame	640 × 480, mirrored
Hand region	300 × 300 pixels at a fixed position
Confidence threshold	0.90
Smoothing	Same gesture in 3 of the last 5 frames
Device	GPU if available, otherwise CPU

11. IMPLEMENTATION

The project is written in Python. The complete source code is submitted separately along with this report. Table 11 lists the main files.

Table 11: Main Source Files

File	Purpose
src/crop_hagrid_hands.py	Builds the dataset from HaGRID
src/data_pipeline.py	Data split, preprocessing and augmentation
src/train_model.py	Builds and trains the CNN
src/evaluate_model.py	Evaluates the model on the test set
src/gesture_recognizer.py	Runs the CNN on webcam frames
src/game_integration.py	Recognition thread and gesture controller
src/play_gesture.py	Main application (game + gesture control)
game/	The maze game: maze, Pac-Man, ghosts, scoring and drawing

To run the project, install the dependencies once with *python install.py* and start the game with *python src/play_gesture.py*. The game can also be played with the keyboard only using *python src/play_gesture.py --no-camera*. On Windows, *setup.bat* and *run_game.bat* do the same by double-clicking.

12. RESULTS AND DISCUSSION

The CNN reached 99.0% accuracy on the test set and 99.6% on people it had never seen, with balanced results across all four gestures. The small gap between the training and validation curves shows that transfer learning, augmentation and dropout kept overfitting low even with only 2,000 images.

The complete application was also tested while running. In a continuous 330-second run with the webcam and GPU, the results in Table 12 were recorded.

Table 12: Real-time Performance

Measure	Result
Game speed	60.1 frames per second
Camera recognition speed	30.4 frames per second
CNN time per frame	18.3 ms on average
Backlog of old predictions	None

Live trials with a person in front of the webcam are summarised in Table 13.

Table 13: Live Webcam Trials

Trial	Result
Holding each gesture (80 trials)	80 of 80 recognised correctly
No hand or relaxed hand (40 trials)	40 of 40 produced no command
Changing from one gesture to another	About 1.3 s on average, mostly the time to move the hand

In addition, 149 automated checks covering the game rules, the integration and the user interface all passed, and we played the game by hand to check directions, turning, ghosts, lives and restarting.

All the objectives were achieved. A balanced dataset was prepared, a CNN was trained and evaluated with very high accuracy, a working maze game was developed, and the CNN controls it in real time through hand gestures while the keyboard remains available.

13. CHALLENGES FACED

- **Dataset images did not match the webcam view.** HaGRID photos show whole scenes, so we cropped every image to the hand. Our first source had pre-cropped images whose bounding boxes no longer matched, so we switched to the resized 384-pixel version.
- **Thumbs-up and thumbs-down are mirror images.** Common augmentations such as vertical flips would have created wrong labels, so we limited augmentation to horizontal flips and small rotations.

- **Avoiding wrong moves.** Acting on every frame would let a single uncertain frame turn Pac-Man the wrong way. The 0.90 threshold and 3-of-5 smoothing, chosen from live measurements, prevent this.
- **Recognition delay.** Recognising a gesture takes a fraction of a second, which is longer than Pac-Man spends at a junction. Keeping each direction request active for 0.35 seconds fixed this; a shorter 0.30-second window still missed a turn in testing.
- **Keeping the game smooth.** Running the CNN inside the game loop would slow the game to the camera's speed, so recognition was moved to a separate thread.

14. LIMITATIONS

- The model only knows four gestures. An unsupported gesture, such as a peace sign, can be wrongly read as one of them.
- Accuracy drops in dark rooms and when the hand moves fast and the image is blurred.
- The hand has to be held inside a fixed box in the camera view.
- Changing from one gesture to another takes about a second.
- Live testing was done with one webcam in one room.

15. FUTURE ENHANCEMENTS

- Add a "no gesture" class so the model can reject unsupported hand shapes.
- Use a hand detector so the hand can be anywhere in the frame.
- Train with darker and blurred images to work better in poor conditions.
- Test with more people, webcams and rooms.
- Make the model smaller and faster for computers without a GPU.

16. CONCLUSION

We developed an interactive game that uses a CNN to recognise hand gestures from a webcam and turn them into game actions. A MobileNetV2 model, adapted through transfer learning on 2,000 HaGRID hand images, reached 99.0% test accuracy and 99.6% on unseen people. Running recognition on a separate thread with a confidence threshold and smoothing let the game stay at 60 frames per second while responding reliably to the player's gestures. The project shows that a compact CNN can give practical real-time game control with just a

webcam. Its main limitation, the lack of a "no gesture" class, is the most useful direction for future work.

17. REFERENCES

[1] J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2009, pp. 248-255.

[2] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov and L.-C. Chen, "MobileNetV2: Inverted Residuals and Linear Bottlenecks," in *Proc. IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 4510-4520.

[3] A. Paszke et al., "PyTorch: An Imperative Style, High-Performance Deep Learning Library," in *Advances in Neural Information Processing Systems 32 (NeurIPS)*, 2019, pp. 8024-8035.

[4] TorchVision maintainers and contributors, "TorchVision: PyTorch's Computer Vision Library," 2016. [Online]. Available: https://github.com/pytorch/vision

[5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[6] Pygame Community, "Pygame 2.6.1." [Online]. Available: https://www.pygame.org

[7] C. R. Harris et al., "Array Programming with NumPy," *Nature*, vol. 585, pp. 357-362, 2020.

[8] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90-95, 2007.

[9] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.

[10] A. Kapitanov, K. Kvanchiani, A. Nagaev, R. Kraynov and A. Makhliarchuk, "HaGRID: HAnd Gesture Recognition Image Dataset," in *Proc. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 2024. arXiv:2206.08219.

[11] C. Mills, "hagrid-sample-500k-384p," Hugging Face Datasets. [Online]. Available: https://huggingface.co/datasets/cj-mills/hagrid-sample-500k-384p

[12] Creative Commons, "Attribution-ShareAlike 4.0 International (CC BY-SA 4.0)." [Online]. Available: https://creativecommons.org/licenses/by-sa/4.0/

[13] S. Ioffe and C. Szegedy, "Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift," in *Proc. International Conference on Machine Learning (ICML)*, 2015, pp. 448-456.

[14] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," *Journal of Machine Learning Research*, vol. 15, pp. 1929-1958, 2014.

[15] I. Loshchilov and F. Hutter, "Decoupled Weight Decay Regularization," in *Proc. International Conference on Learning Representations (ICLR)*, 2019.

[16] P. Micikevicius et al., "Mixed Precision Training," in *Proc. International Conference on Learning Representations (ICLR)*, 2018.